Experiment 3: Java OOP

Inheritance, Encapsulation and Method Overriding

---

1. Question

Write a Java program to demonstrate:

1. Encapsulation using private data members and getter methods.
2. Inheritance using Animal, cow, pig, and horse classes.
3. Method overriding using the sound() method.
4. Display the details of different animals such as name, habitat, food, and sound.

2. Steps

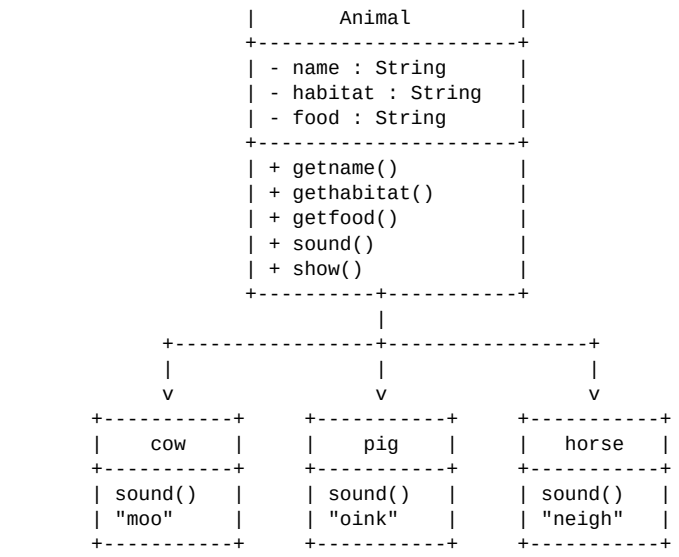
1. Start the program.
2. Create an Animal class.
3. Declare name, habitat, and food as private variables.
4. Create a constructor to initialize these variables.
5. Create getter methods to access the private variables.
6. Create the sound() method in the Animal class.
7. Create the show() method to display animal details.
8. Create cow, pig, and horse classes that extend Animal.
9. Call the parent class constructor using super().
10. Override the sound() method in each child class.
11. Create objects for horse, pig, and cow.
12. Call the show() method for each object.
13. Display the animal details and their sounds.
14. Stop the program.

3. Algorithm

1. Start.
2. Define the Animal class with private variables name, habitat, and food.
3. Initialize these variables using the constructor.
4. Define getter methods for accessing the private variables.
5. Define sound() in the parent class.
6. Define show() to display animal information.
7. Create child classes cow, pig, and horse using inheritance.
8. Override sound() in each child class: Cow → moo, Pig → oink, Horse → neigh.
9. Create objects of horse, pig, and cow.
10. Call show() for each object.
11. Display the corresponding animal information and sound.
12. Stop.

4. UML Diagram

UML Class Diagram



UML Relationship

```
Cow   ----------|> Animal
Pig   ----------|> Animal
Horse ----------|> Animal

This represents inheritance.
```

5. Source Code

```
package packagecodejava;

class Animal {
    private String name;
    private String habitat;
    private String food;

    public Animal(String name, String habitat, String food) {
        this.name = name;
        this.habitat = habitat;
        this.food = food;
    }

    public String getname() {
        return name;
    }

    public String gethabitat() {
        return habitat;
    }

    public String getfood() {
        return food;
    }

    String sound() {
        return "animal sound";
    }

    void show() {
        System.out.println("animal is " + name);
        System.out.println("habitat is " + habitat);
        System.out.println("food is " + food);
        System.out.println("sound is " + sound());
        System.out.println("------------------");
    }
}
```

```
class cow extends Animal {
    public cow() {
        super("cow", "shed", "grass");
    }

    @Override
    String sound() {
        return "moo";
    }
}

class pig extends Animal {
    public pig() {
        super("pig", "sty", "grains,vegetable");
    }

    @Override
    String sound() {
        return "oink";
    }
}

class horse extends Animal {
    public horse() {
        super("horse", "stable", "grass");
    }

    @Override
    String sound() {
        return "neigh";
    }
}

public class Week03 {
    public static void main(String[] args) {
        horse farm = new horse();
        farm.sound();
        farm.show();

        pig far = new pig();
        farm.sound();
        far.show();

        cow fa = new cow();
        fa.sound();
        fa.show();
    }
}
```

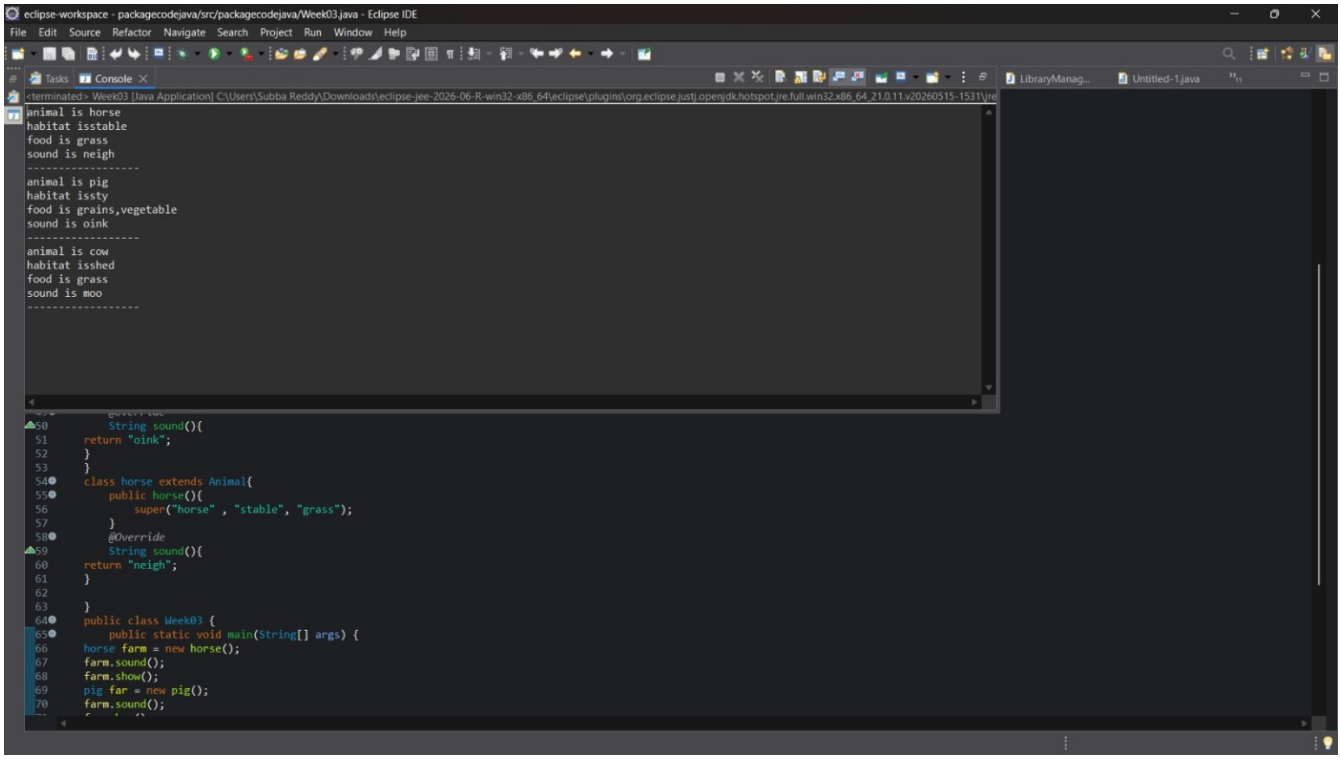
Compilation and Execution

```
javac Week03.java
java packagecodejava.Week03
```

6. Output

```
animal is horse
```

6. Output



Result: The program was executed successfully and the outputs were verified.